TECHNICAL SPECIFICATION

# Rex Latent Decision Engine

An iterative, hypothesis-driven architecture for autonomous incident investigation, threat hunting and infrastructure governance, refactored from the baseline Laya encoder.

October 14, 2026

## AT A GLANCE

**3**
rounds at most per investigation

**3 · 2 · 1**
tool budget K per round, rounds 1, 2 and 3+

**0.35 / 0.15**
score floor and margin for tool gating

**4 + 1**
MECE hypotheses plus the residual H_novel

Sources: Rex architectural specification (refactored from Laya).

# Contents

1 System Overview . . . . . 3

2 Architecture and Runtime Flow . . . . . 4

3 Context Hub, Input Formatter and Encoder Backbone . . . . . 4

4 Multi-Tool Selection Pipeline . . . . . 5

5 Execution Orchestrator and Client SDK . . . . . 6

6 PoE Engine, Convergence Evaluator and Verdict Head . . . . . 7

7 Data Contracts and Pipeline Interfaces . . . . . 8

8 Verification and Operational Runbook . . . . . 10

9 Risks, Open Items and Calibration Notes . . . . . 11

# 1 System Overview

Rex is a non-autoregressive latent decision engine for incident investigation, threat hunting and infrastructure governance. It replaces the single-turn token classifier of the baseline Laya encoder with an iterative, hypothesis-driven loop of at most three rounds.

Each round follows the same sequence: the encoder reads the current state, a bounded set of tools runs in parallel, a falsification engine eliminates hypotheses the data contradicts, and a convergence check decides whether to continue or issue a verdict. The verdict is a softmax over surviving hypotheses only, so eliminated causes can never be returned.

## Core problems solved

| Vulnerability in baseline Laya | Mechanism in Rex |
|---|---|
| **Probability cannibalization.** Softmax over tool outputs is zero-sum and suppresses parallel tools that are all relevant. | **Independent sigmoid scoring.** Each tool is scored as its own Bernoulli event. |
| **Avalanche activation.** A small positive latent shift pushes 5 to 10 collinear tools past a static 0.5 threshold. | **Dynamic relative margin gating plus a descending budget cap.** Only competitive runners-up pass, and output size is hard-capped at $K_t$. |
| **Hallucinated root causes.** Generative output invents unverified failure modes. | **Closed-set hypothesis tracking with PoE masking.** Elimination is deterministic, with a hard logit of negative infinity. |
| **Closed-world failure.** Unseen errors and zero-day incidents have no class to land in. | **Hierarchical MECE taxonomy plus a residual slot ($H_{novel}$).** The state space is exhaustive by construction. |

# 2 Architecture and Runtime Flow

An alert is parsed once, then Rex repeats a six-step loop for at most three rounds before the Verdict Head decides.

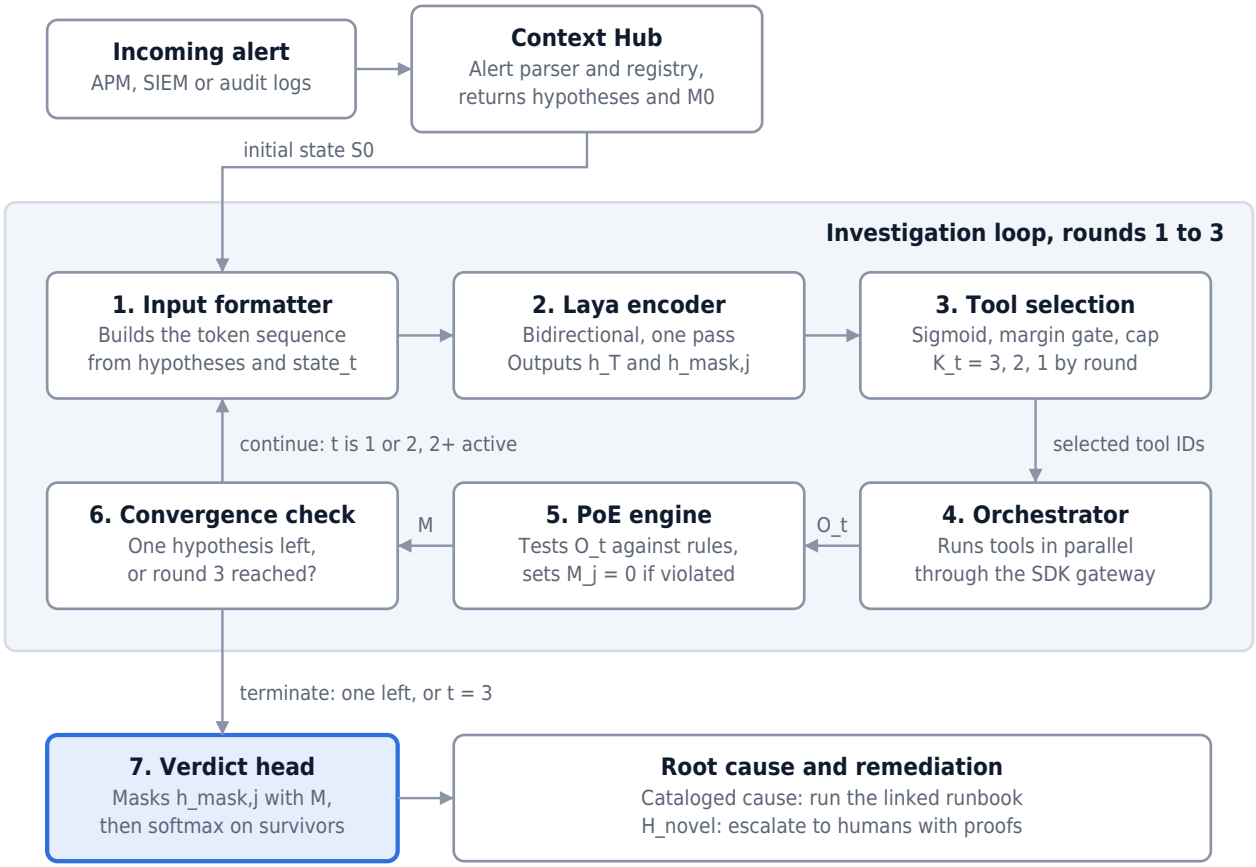



*Figure 1. Rex investigation loop: 7 steps, 1 loop-back*

The Context Hub hands the formatter an initial state $S_0$. In each round the encoder produces $h_T$ for tool dispatch and $h_{mask,j}$ for the verdict. After the PoE engine updates the mask M, the convergence check either appends the observations to the state and re-enters at step 1, or routes to the Verdict Head, which reads $h_{mask,j}$ and M directly.

# 3 Context Hub, Input Formatter and Encoder Backbone

These three components set up each round: the Hub creates the investigation once, the Formatter builds the token sequence every round, and the encoder turns it into two kinds of vectors.

## Component 1: Pre-Execution Context Hub and Registry

The Hub initializes the investigation when an external trigger arrives, such as an alert payload or a telemetry spike.

- **Input:** raw alert JSON from APM, SIEM or audit-log systems.
- **Logic:** an Alert Classifier and Normalizer maps the alert to a domain token `<type>` (for example `TYPE_LATENCY`) by exact or indexed lookup. The Hypothesis and Tool Registry then returns the MECE hypothesis set and tool metadata.
- **Output:** the initial state object `S_0 = { type, hypotheses, M_0, state_0 }`.

The fixed hypothesis set is H = {$H_0$, $H_1$, ..., $H_{M-1}$, $H_{novel}$}. The initial mask is all ones, with M + 1 entries:

$$\mathbf{M}_0 = [1, 1, \ldots, 1]^T \in \{0, 1\}^{M+1}$$

### Component 2: Input Tensor Formatter

The Formatter builds the exact token sequence the encoder expects at each round t.

```
[CLS] <type> instructions [SEP] [MASK] H_0 [MASK] H_1 ... [MASK] H_novel [SEP] state_t [SEP]
```

| Token | Role |
|---|---|
| `[CLS]` | Position 0. Its final representation $h_T$ (a d-dimensional vector) carries the global state and is used only for tool dispatch. |
| `<type>` | Domain identifier, such as <TYPE_API_LATENCY>. |
| `[MASK] H_j` | Each hypothesis string gets its own `[MASK]` prefix. Bidirectional attention lets `[MASK]_j` read its own description and the evidence in `state_t`. |
| `state_t` | Running log of observations across rounds. |

The state accumulates as a concatenation of the initial state and every prior round's observations:

$$\text{state}_t = [\text{state}_0 \,\|\, \mathcal{O}_1 \,\|\, \ldots \,\|\, \mathcal{O}_{t-1}]$$

### Component 3: Laya Transformer Encoder Backbone

The backbone is an invariant bidirectional Transformer encoder. It processes the full sequence in one pass, with no autoregressive decoding.

$$\mathbf{H} = \text{TransformerEncoder}(\text{Tokens}) \in \mathbb{R}^{L \times d}$$

Two slices of the output are routed separately:

- **$h_T$ = H[0]** goes to the Multi-Tool Selection Pipeline.
- **$h_{mask,j}$ = H[pos([MASK]_j)]** goes to the Verdict Head.

# 4 Multi-Tool Selection Pipeline

The pipeline turns the latent vector $h_T$ into a bounded list of tools to run in parallel. A tool projection head (an MLP) produces raw logits $z_{tool}$, which then pass through three layers in order.

| Layer | Operation | Parameters |
|---|---|---|
| **1.** PoE invalidation and sigmoid scoring | Set $z_i$ to negative infinity for any tool that depends strictly on a hypothesis with $M_j = 0$, then score each tool independently. | None |
| **2.** Dynamic relative margin gating | Sort scores descending. Keep tool i only if it clears the floor and sits close to the leader. | $s_{min} = 0.35$, $\Delta_{margin} = 0.15$ |
| **3.** Descending budget cap | Truncate the valid set to the top $K_t$ tools. | $K_1 = 3$, $K_2 = 2$, $K_{3+} = 1$ |

### Layer 1: PoE masking and sigmoid scoring

A tool whose supporting hypothesis has been eliminated receives a logit of negative infinity, so its score is exactly zero. Every other tool is scored independently, which removes the zero-sum competition of softmax.

$$z_i \leftarrow -\infty \;\Rightarrow\; s_i = \sigma(z_i) = 0.0$$

$$s_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}} \quad \forall\, i \in \{1, \ldots, N\}$$

### Layer 2: Dynamic relative margin gating

With sorted scores $s_1 \geq s_2 \geq \ldots \geq s_N$, the valid set contains tools that clear the absolute floor and stay within the margin of the top score. If $s_1 < s_{\min}$, the valid set is empty.

$$\mathcal{T}_{\text{valid}} = \left\{ i \mid s_i \geq s_{\min} \;\wedge\; (s_1 - s_i) \leq \Delta_{\text{margin}} \right\}$$

### Layer 3: Descending budget cap

The executed set is the top $K_t$ of the valid set. The cap shrinks each round, which forces the investigation toward fewer, more decisive probes.

$$\mathcal{T}_{\text{exec}} = \text{Top-}K_t(\mathcal{T}_{\text{valid}}), \qquad K_t = 3\ (t=1),\ \ 2\ (t=2),\ \ 1\ (t \geq 3)$$

The output is the tool execution dispatch list. An empty list sends the loop straight to the Verdict Head.

# 5 Execution Orchestrator and Client SDK

The orchestrator connects the model's discrete tool choices to the customer's real environment through an outbound WebSocket client, compatible with the Model Context Protocol (MCP). The customer side always initiates the connection, so no inbound firewall or VPC changes are needed.

1. The SDK client on the developer host opens an outbound TLS (WSS) tunnel to the Rex Gateway and syncs the available tool schemas.
2. For each selected tool, the Rex engine in the cloud sends a JSON-RPC 2.0 `tools/call` request.
3. On the host, the SDK runs a zero-mutation check, redacts PII locally, and executes the tool handler.
4. Clean observations return to the engine over the same tunnel.

**Concurrency.** Selected tools run in parallel with `asyncio.gather(*[tool.run() for tool in selected])`.

### Trust and safety boundary

| Control | Behavior |
|---|---|
| Read-only enforcement | Tools default to read-only semantics. Any write operation triggers an explicit local authorization prompt. |
| Local redaction | Credentials, authorization headers, private keys and environment variables are scrubbed by the SDK before payloads leave host memory. |
| Network direction | Outbound TLS only, which avoids corporate firewalls, NAT and private VPC ingress restrictions. |

# 6 PoE Engine, Convergence Evaluator and Verdict Head

These components decide which hypotheses survive, when to stop, and what the final answer is. Hypotheses are never proven in the early rounds. They can only survive elimination.

## Component 6: Process of Elimination (PoE) Falsification Engine

The engine applies Karl Popper's falsification principle. It compares each new observation set $O_t$ against the necessary symptoms of every active hypothesis.

- **Eliminate ($M_j = 0$)** if and only if the data contradicts a necessary symptom of $H_j$.
- **Keep ($M_j = 1$)** if the data is consistent with the hypothesis.
- **Residual slot:** $H_{novel}$ has no static falsification rules and stays active as long as an anomaly exists.

Worked example with observation `{db_connections: 12.5, cpu_usage: 94.2}`:

| Hypothesis | Falsification rule | Evaluation | Result |
|---|---|---|---|
| $H_0$: DB connection pool exhaustion | $db_{connections} < 80\%$ | 12.5 < 80, violation found | $M_0 = 0$ |
| $H_1$: CPU throttling on pods | $cpu_{usage} < 70\%$ | 94.2 ≥ 70, consistent | $M_1 = 1$ |
| $H_{novel}$: uncataloged root cause | Protected slot | Not evaluated | $M_{novel} = 1$ |

The resulting mask in this example is $M_t = [0, 1, 1, 1, 1]$.

## Component 7: Convergence Evaluator and State Accumulator

The loop ends when either condition holds; otherwise it continues.

1. **Singleton convergence:** exactly one hypothesis remains active.
2. **Budget depletion:** the round count reaches t = 3, or the tool pipeline returned an empty execution set.

$$\sum_j M_j = 1$$

On termination, execution goes to the Verdict Head. On continuation, the engine appends $O_t$ to $state_t$ to form $state_{t+1}$, increments t, and re-invokes the Input Tensor Formatter.

### Component 8: Verdict Head and Hard Logit Masking

The Verdict Head projects each `[MASK]_j` hidden state to a scalar logit, masks out eliminated hypotheses, and applies softmax over the survivors.

$$z'_{\text{verdict},j} = z_{\text{verdict},j} \;\text{ if } M_j = 1, \qquad z'_{\text{verdict},j} = -\infty \;\text{ if } M_j = 0$$

$$P(H_j \mid \mathbf{H}, \mathbf{M}) = \frac{\exp(z'_{\text{verdict},j})}{\sum_{m=0}^{M} \exp(z'_{\text{verdict},m})}$$

The decision path depends on the argmax:

- **Cataloged hypothesis $H_k$ (k < M):** trigger the linked automated remediation runbook.
- **$H_{novel}$:** trigger the Human Escalation Workflow, exporting the full telemetry trail and elimination proofs to incident responders.

# 7 Data Contracts and Pipeline Interfaces

Two artifacts define the contract: a YAML hypothesis registry that the Context Hub loads, and a PyTorch module that implements tool selection and the verdict.

## 7.1 Hypothesis registry schema (YAML)

Each hypothesis carries its associated tools and its falsification rules. The residual hypothesis sets `is_residual: true` and has no rules.

```
type: TYPE_API_LATENCY_SPIKE
description: "P99 latency exceeding service level objectives (> 500ms)"
hypotheses:
  - id: "H_infra"
    text: "Host level resource exhaustion such as CPU throttling or memory starvation"
    associated_tools: ["get_system_metrics", "inspect_cgroup_limits"]
    falsification_rules:
      - metric: "cpu_usage_percent"
        operator: "LT"
        threshold: 70.0
      - metric: "memory_swap_usage"
        operator: "EQ"
        threshold: 0.0

  - id: "H_app"
    text: "Application runtime deadlock, garbage collection stall, or unhandled errors"
    associated_tools: ["dump_thread_traces", "get_gc_statistics"]
    falsification_rules:
      - metric: "jvm_gc_pause_ms"
        operator: "LT"
        threshold: 100.0
      - metric: "blocked_thread_count"
        operator: "EQ"
        threshold: 0.0

  - id: "H_dep"
    text: "Downstream external dependency or database connection pool failure"
    associated_tools: ["check_db_pool", "ping_downstream_gateways"]
    falsification_rules:
      - metric: "db_active_pool_percent"
        operator: "LT"
        threshold: 80.0
      - metric: "downstream_5xx_rate"
        operator: "EQ"
        threshold: 0.0

  - id: "H_data"
```

```
    text: "Ingress payload anomaly, queue poisoning, or traffic volume surge"
    associated_tools: ["inspect_queue_depth", "analyze_request_rates"]
    falsification_rules:
      - metric: "rps_surge_ratio"
        operator: "LT"
        threshold: 1.2

  - id: "H_novel"
    text: "Uncataloged anomaly or cascading zero-day failure mode"
    is_residual: true
    associated_tools: ["capture_raw_packet_sample", "inspect_audit_trail"]
    falsification_rules: []
```

## 7.2 PyTorch runtime module

`RexDecisionPipeline` holds the tool MLP and the shared verdict projection. `select_tools` implements layers 1 to 3 of the selection pipeline, and `compute_verdict` implements hard logit masking with softmax.

```
"""Rex Decision Engine: Core Decision and Tool Pipeline Implementation."""

from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
import torch
import torch.nn as nn
import torch.nn.functional as F


@dataclass
class HypothesisDefinition:
  id: str
  text: str
  associated_tools: List[int]
  is_residual: bool = False
  falsification_criteria: List[Dict[str, Any]] = field(default_factory=list)


class RexDecisionPipeline(nn.Module):
  """End-to-End Decision Head for the Rex Agentic Engine."""

  def __init__(
      self,
      hidden_dim: int,
      num_tools: int,
      s_min: float = 0.35,
      delta_margin: float = 0.15,
  ):
    super().__init__()
    self.hidden_dim = hidden_dim
    self.num_tools = num_tools
    self.s_min = s_min
    self.delta_margin = delta_margin

    # Tool Selection MLP Head
    self.tool_mlp = nn.Sequential(
        nn.Linear(hidden_dim, hidden_dim // 2),
        nn.LayerNorm(hidden_dim // 2),
        nn.GELU(),
        nn.Linear(hidden_dim // 2, num_tools),
    )

    # Shared Verdict Projection Head
    self.verdict_projection = nn.Linear(hidden_dim, 1)

  def select_tools(
      self,
      h_cls: torch.Tensor,
      round_idx: int,
      tool_mask: Optional[torch.Tensor] = None,
  ) -> List[int]:
    """Selects parallel executable tools using Sigmoid + Margin Gating + Budget Cap.

    Args:
        h_cls: Tensor [hidden_dim] at token [CLS].
        round_idx: Current turn count (1-indexed).
        tool_mask: Optional binary tensor [num_tools] (0 = branch falsified).

    Returns:
```

```
        List of approved tool indices.
    """
    # Determine descending budget cap K_t
    budget_cap = 3 if round_idx == 1 else (2 if round_idx == 2 else 1)

    logits = self.tool_mlp(h_cls)

    # Apply PoE tool mask
    if tool_mask is not None:
      logits = logits.masked_fill(tool_mask == 0, -float("inf"))

    # Layer 1: Independent Sigmoid Scoring
    scores = torch.sigmoid(logits)

    # Layer 2: Dynamic Margin Gating
    sorted_scores, sorted_indices = torch.sort(scores, descending=True)
    top_1_score = sorted_scores[0].item()

    if top_1_score < self.s_min or torch.isinf(sorted_scores[0]):
      return []

    selected_tool_indices = [sorted_indices[0].item()]

    # Scan runners-up within budget
    max_scan = min(budget_cap, len(sorted_scores))
    for i in range(1, max_scan):
      score_i = sorted_scores[i].item()
      if (
          score_i >= self.s_min
          and (top_1_score - score_i) <= self.delta_margin
      ):
        selected_tool_indices.append(sorted_indices[i].item())
      else:
        break

    return selected_tool_indices

  def compute_verdict(
      self, h_masks: torch.Tensor, poe_mask: torch.Tensor
  ) -> torch.Tensor:
    """Computes final root-cause probabilities across candidate hypotheses.

    Args:
        h_masks: Tensor [num_hypotheses, hidden_dim] at [MASK]_j tokens.
        poe_mask: Binary tensor [num_hypotheses] where 0 denotes elimination.

    Returns:
        Tensor of probabilities summing to 1.0 across active hypotheses.
    """
    logits = self.verdict_projection(h_masks).squeeze(-1)  # [num_hypotheses]
    masked_logits = logits.masked_fill(poe_mask == 0, -float("inf"))
    return F.softmax(masked_logits, dim=-1)
```

# 8 Verification and Operational Runbook

The walkthrough below exercises every component once. Run it with a mock incident of type `TYPE_API_LATENCY_SPIKE`.

## 8.1 Step-by-step execution verification

1. **Trigger:** send a mock incident payload with `type = TYPE_API_LATENCY_SPIKE`.
2. **Round 1:**
   - Confirm the formatted input contains 4 MECE hypotheses and 1 $H_{novel}$ slot.
   - Confirm the tool MLP selects at most 3 tools ($K_1 \leq 3$).
   - Confirm `get_system_metrics` and `check_db_pool` both run in parallel through `asyncio.gather`.
3. **PoE invalidation:** feed the observation `{"cpu_usage_percent": 14.0, "db_active_pool_percent": 96.0}`.
   - Confirm $H_{infra}$ is falsified ($M_{infra} = 0$) because `cpu_usage_percent < 70.0`.
   - Confirm $H_{dep}$ stays active ($M_{dep} = 1$).

4. **Round 2:**
   - Confirm the budget cap drops to $K_2 \leq 2$.
   - Confirm tools exclusive to $H_{infra}$ get logit negative infinity and are not dispatched.
5. **Verdict:** feed a final observation confirming database connection saturation.
   - Confirm $P(H_{dep}) > 0.95$ and $P(H_{infra}) = 0.0$.

### 8.2 Failure modes and recovery

| Symptom | Underlying fault | Immediate remediation |
|---|---|---|
| All known hypotheses eliminated in round 1 ($M_0$ to $M_{M-1}$ all 0) | Out-of-distribution event or conflicting telemetry | The engine keeps $M_{novel} = 1$, short-circuits the loop, and triggers Human Escalation with the full debug trace. |
| Tool execution timeout in the client SDK | Host process unresponsive, or network partition in the client VPC | The orchestrator logs a partial observation `{tool_status: TIMEOUT}`, drops the tool from margin selection, and moves to the next round without crashing. |
| Top score falls below $s_{min}$ (< 0.35) | Too little signal in `state_t` to justify tool cost | The pipeline returns an empty tool list and short-circuits to the Verdict Head to avoid useless API spend. |

# 9 Risks, Open Items and Calibration Notes

The gating thresholds are the main known risk: $s_{min} = 0.35$ and $\Delta_{margin} = 0.15$ must be recalibrated during post-training fine-tuning if the backbone size changes, for example when scaling from 110M to 350M parameters.

The items below are gaps noticed while turning the source spec into this document. They are questions for the spec owner, not changes to the design.

| Open item | Why it matters |
|---|---|
| **Rule combination in the YAML registry.** $H_{infra}$ and $H_{app}$ each list two falsification rules, but the spec does not say whether one violated rule or all of them eliminate a hypothesis. | Under "any rule", high CPU with zero swap would eliminate $H_{infra}$ even though CPU alone supports it. This conflicts with the contract that elimination requires contradicting necessary symptoms. |
| **Rule direction.** Rules such as `cpu_usage_percent LT 70.0` read as the elimination condition, not the symptom. | The registry format should state this explicitly, since the opposite reading inverts every elimination. |
| **Tool identifiers.** The YAML lists tools by name, while `HypothesisDefinition.associated_tools` is a list of integer indices. | A name-to-index mapping is needed to build `tool_mask` from the PoE mask. |
| **Shared tools.** A tool is masked only if it depends strictly on an eliminated hypothesis, but the code takes `tool_mask` as an input and does not derive it. | The derivation rule for tools shared across hypotheses should be specified. |
| **Verdict threshold.** The verification step expects $P(H_{dep}) > 0.95$, but no calibration procedure for the verdict head is given. | The 0.95 figure is a test expectation that depends on training, not a guaranteed property of the architecture. |